ТИИСПИС · БГУИР · КАФ. ИИТ

Защита ЛР1 по ТиИСПИС

19 вопросов преподавателя, разбор задач и справочник по SCs. Вариант №26 — «Языки программирования».

ЭТО НАДО НАЗВАТЬ НЕ ЗАДУМЫВАЯСЬ

Вариант	**№26 «Языки программирования»**
Главный максимальный класс	`concept_programming_language`
Понятий	**23 абсолютных + 17 относительных = 40**
Из них отношений	**14 неролевых + 3 ролевых**
Классы в структуре ПрО	**10 максимальных, 13 немаксимальных**
Экземпляров	**40, связаны 70 связями**
Файлов `.scs`	**77**
Разбиений `nrel_subdividing`	**5**

Вопросы к защите

Три балла из десяти. У каждого вопроса — ответ, что сказать вслух и чем подкрепить из своей работы.

01 Что такое Технология проектирования интеллектуальных систем? Какие цели ставит и какие задачи решает?

Технология — комплекс моделей, методов и средств, обеспечивающих решение соответствующего класса задач. Проектирование технологии сопровождается разработкой стандарта.

Технология OSTIS (Open Semantic Technology for Intelligent Systems) — комплекс технологий, обеспечивающих проектирование, производство, эксплуатацию и реинжиниринг интеллектуальных компьютерных систем. В основе — смысловое представление и онтологическая систематизация знаний, агентно-ориентированная обработка знаний.

Цели и задачи, дословно две: разработка ostis-систем, семантически совместимых между собой, способных к самостоятельному взаимодействию и адаптации к пользователям; интеграция и конвергенция различных видов знаний и моделей решения задач.

> СКАЗАТЬ ВСЛУХ
>
> Ключевые слова: семантическая совместимость, конвергенция и интеграция знаний. Конвергенция — стремление делать одинаково то, что можно сделать одинаково, без ущерба качеству. Интеграция — объединение разного в одно целое. Чем выше конвергенция, тем качественнее интеграция.

02 Какие виды интеллектуальных систем Вы знаете?

- **интеллектуальная справочная** — имеет средства навигации по материалу и задания вопросов по нему;
- **интеллектуальная обучающая** — обязательно реализует функции справочной, плюс обучает;
- **интеллектуальная экспертная** — содержит знания об узкой слабо структурированной и трудно формализуемой области, способна предлагать и **объяснять** решения;
- системы поддержки принятия решений, диалоговые ассистенты, многоагентные и робототехнические системы.

Между первыми тремя есть теоретико-множественная связь: каждая обучающая система является справочной, то есть обучающие — подмножество справочных по набору возможностей.

> ИЗ МОЕЙ РАБОТЫ
>
> NIKA — обучающая экспертная система, то есть совмещает все три вида сразу.

03 Что является объектом изучения Искусственного интеллекта? Какие направления известны?

Искусственный интеллект — научно-техническая деятельность, направленная на построение теории интеллектуальных систем и на создание технологии проектирования и производства искусственных интеллектуальных систем.

Три цели дословно: построение теории интеллектуальных систем; создание технологии разработки ИКС; переход на принципиально новый уровень комплексной автоматизации всех видов человеческой деятельности.

Третья цель предполагает не только наличие систем, понимающих друг друга, но и построение **Общей теории человеческой деятельности** — теории деятельности smart-общества, понятной самим системам.

Направления: представление знаний, решение задач, обучение, обработка естественного языка, распознавание образов, многоагентные системы, семантические технологии.

> СКАЗАТЬ ВСЛУХ
>
> Методичка подчёркивает: проектирование технологии ведётся едино, по всем направлениям ИИ. Наш курс — направление представления знаний и семантических технологий.

04 Что такое интеллектуальная компьютерная система? Какими свойствами должна обладать?

ИКС — искусственная кибернетическая система, обладающая высоким уровнем интеллекта, то есть высоким уровнем знаний и умений, а также высоким уровнем **обучаемости**.

Частный случай — **ostis-система**, построенная по требованиям и стандартам Технологии OSTIS. Это обеспечивает семантическую совместимость всех таких систем между собой и способность к самостоятельному взаимодействию при возникновении заранее непредсказуемых, нештатных ситуаций.

Свойства: наличие базы знаний; обучаемость; понимание сообщений и обмен информацией с другими системами; обоснование и объяснение решений; координация действий при коллективном решении задач; рефлексия — знания о самой себе.

> СКАЗАТЬ ВСЛУХ
>
> Сильный аргумент из методички: все обстоятельства работы систем заранее предусмотреть принципиально невозможно, поэтому без этих способностей никакие smart-предприятия и smart-общество невозможны.

05 В чём отличие интеллектуальной компьютерной системы от искусственной интеллектуальной системы?

По тексту методички это синонимы. Там написано буквально: «Под любой интеллектуальной компьютерной системой *(искусственной интеллектуальной системой)* понимается искусственная кибернетическая система...» — термины даны через скобки как равнозначные.

Разница только в акценте: «компьютерная» указывает на реализацию средствами вычислительной машины, «искусственная» — на противопоставление естественному интеллекту человека.

> СКАЗАТЬ ВСЛУХ
>
> Если преподаватель ждёт различия — допустима трактовка через включение: искусственная интеллектуальная система шире, сюда попадают и гибридные «человек — машина», а компьютерные являются её подмножеством. Но сначала скажи, что в методичке это дано как синонимы: вопрос проверяет внимательность к тексту.

06 Из каких компонентов состоит любая ostis-система? Возможно ли существование без какого-то компонента?

Три компонента: **база знаний**, **решатель задач**, **интерфейс**.

Нет, невозможно. Компоненты отвечают за три необходимые функции: хранение знаний, их обработку и обмен с внешней средой. Без базы знаний решателю нечего обрабатывать. Без решателя система — пассивное хранилище, задач не решает. Без интерфейса нельзя ни поставить задачу, ни получить результат, то есть система внешне неотличима от несуществующей.

> ИЗ МОЕЙ РАБОТЫ
>
> Наглядный пример: я наполнил базу знаний, она видна в sc-web. Но в диалоге система не отвечает — не отрабатывает часть решателя, отвечающая за классификацию сообщений. Знания есть, функции нет. Это и показывает, что компоненты неразделимы.

07 Что такое база знаний ostis-системы? Какие знания можно в ней представлять?

Методичка даёт **два определения**. Первое, через достаточность: совокупность знаний, хранимых в памяти ИКС и достаточных для того, чтобы система удовлетворяла предъявляемым к ней требованиям, в частности имела соответствующий уровень интеллекта.

Второе, через качество: информация, имеющая высокий уровень по пяти показателям — **семантическая мощность** языка представления, **гибридность**, **многообразие видов знаний**, **формализованность**, **структурированность**.

Виды знаний: фактографические высказывания; логические формулы, определения и правила вывода; онтологии и предметные области; спецификации sc-агентов; знания об интерфейсе; документация самой системы; метазнания.

> **СКАЗАТЬ ВСЛУХ**
>
> Главное: в базе знаний хранится не только предметная информация, но и описание самой системы. Отсюда рефлексия.

08 Что такое SC-код? Какие задачи решаются с его помощью? Приведите фрагмент.

SC-код — формальный язык универсального представления знаний в памяти интеллектуальных компьютерных систем; представляет собой множество **нелинейных sc-текстов**.

Ключевая особенность: тексты SC-кода — графовые структуры расширенного вида, в которых знаки связей могут соединять не только вершины, но и **знаки других связей**.

Какие задачи решает: единая форма записи разных видов знаний — и декларативных, и процедурных. На SC-коде можно записать любую информацию, в том числе программу, обрабатывающую другую информацию. Отсюда следует, что память ostis-системы **графодинамическая** — нелинейная и структурно перестраиваемая.

Внешние языки: **SCs** — текстовый, **SCg** — графический, **SCn** — для чтения человеком.

```
python_language
<- concept_programming_language;
<- concept_interpreted_programming_language;
=> nrel_main_idtf:
    [Python] (* <- lang_ru;; *);
=> nrel_language_author: guido_van_rossum;
=> nrel_creation_year: year_1991;;
```

> **ИЗ МОЕЙ РАБОТЫ**
>
> Это реальный экземпляр из моей предметной области. Видно принадлежность двум классам, идентификатор с указанием языка и две связки по неролевым отношениям.

09 Что такое знание? Приведите пример.

Знание — дискретная информационная конструкция, представленная в некотором языке. **Sc-знание** — знание, представленное на SC-коде.

Не каждый sc-текст является знанием: в знании важна **семантическая целостность, полнота** информационной конструкции.

Контрпримеры прямо из методички: логическая формула со свободными переменными знанием не является; sc-текст, в состав которого входит коннектор или связка, но не входят все их компоненты, знанием не является.

> ИЗ МОЕЙ РАБОТЫ
>
> «Си++ создан под влиянием языка Си» — знание: все компоненты связки на месте, обе сущности принадлежат известным классам, связь принадлежит отношению с заданными доменами.

10 В чём отличие знания от информации? Приведите пример и обоснуйте.

Информация — сведения в некоторой форме; сами по себе они не обязаны быть интерпретируемы системой.

Знание — информация, обладающая семантической целостностью и погружённая в систему понятий: у неё однозначная интерпретация, она связана с другими знаниями, и на ней возможен вывод.

> ИЗ МОЕЙ РАБОТЫ
>
> Строка `1972` сама по себе — информация, непонятно что за число. Конструкция «Си ⇒ год создания*: 1972 год», где `year_1972` принадлежит классу «календарный год», — уже знание: система может ответить на вопрос о годе создания Си и связать его с другими языками того же периода.

> СКАЗАТЬ ВСЛУХ
>
> Ключ различия — наличие семантической интерпретации и связей, а не форма записи. Формально: у знания есть все компоненты связки, у обрывка информации их нет.

11 Какими достоинствами база знаний наделяет ostis-систему?

Высокий уровень обучаемости обеспечивается тремя свойствами:

- **семантическая гибкость** — каждое добавление или удаление элементарного фрагмента и каждой связи инцидентности имеет чёткую семантическую интерпретацию;
- **стратифицированность** хранимой информации — за счёт онтологически ориентированной структуризации;
- **рефлексия** — за счёт мощных метаязыковых возможностей языка внутреннего представления.

Отдельно методичка перечисляет, чем обеспечиваются достоинства ostis-систем в целом: достоинствами SC-кода, организации sc-памяти, sc-моделей баз знаний и sc-моделей решения задач.

12 Как организуется и структурируется база знаний? Приведите пример разделов и предметных областей.

База знаний — **иерархическая система разделов**, к которым привязываются исходные тексты новой информации. Разделы бывают **атомарными** — без сегментов, и **неатомарными** — состоящими из сегментов.

В основе семантической структуризации лежит **иерархическая система предметных областей**; каждой ПрО обычно соответствует свой раздел. У раздела указываются ключевые sc-элементы через `rrel_key_sc_element`.

Смысл — «семантические полочки»: знания лежат там, где их логично искать, и легко определяется место для новых.

> **ИЗ МОЕЙ РАБОТЫ**
>
> Раздел `section_subject_domain_of_programming_languages` объявлен атомарным, внутри его контура описана структура `subject_domain_of_programming_languages` с 10 максимальными классами, 13 немаксимальными и 17 исследуемыми отношениями. Рядом в базе NIKA лежат ПрО сообщений, диалогов, действий и задач.

13 Что такое система NIKA? Какими функциями обладает? Какие цели преследует?

NIKA — интеллектуальная обучающая экспертная система, интеллектуальный ассистент, управляемый знаниями (*NIKA is an Intelligent Knowledge-driven Assistant*). Является ostis-системой.

Состав: гибридная база знаний, гибридный решатель задач, интеллектуальный интерфейс.

Что описано в её базе знаний: документация; предметные области с понятиями для решения логических задач и управления диалогом; логические правила построения диалога; агенты управления диалогом и их спецификация; прикладные ПрО, на тему которых ведётся диалог.

Функции: отвечать на приветствие; отвечать на вопросы о сущности, о разбиении, о навыках; отвечать про условия и сроки сдачи лабораторных; искать слова на заданную букву; менять цвет компонентов интерфейса; обучаться новым вопросам.

Цели: расширить круг решаемых задач и устранить недостатки ранее использованного аналога; упростить и автоматизировать обучение студентов; обучить студентов специальности «Искусственный интеллект» БГУИР технологии проектирования интеллектуальных систем.

> **СКАЗАТЬ ВСЛУХ**
>
> Быстрая проверка, что система жива: отправить пустое сообщение — Ника выдаст список того, что умеет.

14 Что такое абсолютное и относительное понятие? Объясните разницу и критерии выбора.

Понятие — класс сущностей, входящий в состав хотя бы одной предметной области в роли ключевого исследуемого понятия.

Абсолютное понятие — класс, множество элементов с явно указываемыми общими свойствами. Конкретный элемент класса называется **экземпляром**. Префикс `concept_`.

Относительное понятие — отношение, то есть класс связок между объектами; формально — подмножество декартова произведения. Отношения делятся на бинарные и небинарные, ориентированные и неориентированные, **ролевые** (`rrel_`, апостроф) и **неролевые** (`nrel_`, звёздочка).

Критерий выбора: если сущность имеет смысл только как связь между двумя и более другими сущностями — она относительная; если её можно осмыслить саму по себе — абсолютная. Проверочный вопрос: «что она связывает?» Если ответить нечем — понятие абсолютное.

> ИЗ МОЕЙ РАБОТЫ
>
> «язык программирования», «компилятор» — абсолютные. «автор языка программирования*», «создан под влиянием языка*» — относительные: каждое связывает две сущности, и для каждого заданы первый и второй домены.

15 Из каких основных частей состоит предметная область?

Формально ПрО — результат интеграции частичных семантических окрестностей, описывающих исследуемые сущности заданного класса с одинаковым набором отношений. В состав входят:

- основные исследуемые объекты — первичные и вторичные;
- различные классы исследуемых объектов;
- связки, компонентами которых являются исследуемые объекты, а возможно и другие связки — то есть связки имеют разный структурный уровень;
- классы этих связок, то есть **исследуемые отношения**;
- классы объектов, не являющихся ни объектами исследования, ни связками, но входящих в состав связок.

Требование: все объявленные классы должны быть полностью представлены вместе со своими элементами, элементами элементов и так далее вплоть до терминальных.

> ИЗ МОЕЙ РАБОТЫ
>
> Последний пункт у меня — «календарный год»: годы не изучаются сами по себе, но входят в связки с языками через отношение «год создания*».

16 В чём различие максимального класса объектов исследования и класса объектов исследования? Пример.

Максимальный класс — тот, для которого в заданной предметной области **отсутствует другой класс объектов исследования, являющийся его надмножеством**. Немаксимальный — любой изучаемый класс, входящий в максимальный или описывающий компоненты изучаемых объектов.

> ИЗ МОЕЙ РАБОТЫ
>
> Максимальный — `concept_programming_language`. Немаксимальные — `concept_compiled_programming_language` как подкласс и `concept_compiler` как компонент окружения. В структуре ПрО это разные ролевые отношения: `rrel_maximum_studied_object_class` и `rrel_not_maximum_studied_object_class`.

> СКАЗАТЬ ВСЛУХ
>
> Тонкость, которую стоит проговорить самому: максимальность задана **относительно конкретной ПрО**, а не абсолютно. У меня язык программирования включён в «формальный язык», но формальный язык в этой ПрО не исследуется, поэтому язык программирования остаётся максимальным.

17 Приведите пример теоретико-множественных отношений между понятиями различного вида. Зачем их указывать?

Виды: принадлежность (`<-`), включение (`nrel_inclusion`), строгое включение, разбиение (`nrel_subdividing`), пересечение, объединение, равенство множеств.

Зачем указывать:

- позволяют системе **выводить новые знания** — свойства надкласса наследуются подклассом;
- делают базу знаний **навигируемой** — по ним строится структура;
- позволяют **обнаруживать противоречия**, например цикл во включении;
- разбиение — более сильное утверждение, чем включение: оно дополнительно говорит, что подклассы не пересекаются и покрывают надкласс.

> ИЗ МОЕЙ РАБОТЫ
>
> У понятия «язык программирования» пять разбиений: по способу выполнения, по уровню абстракции, по дисциплине типизации и по основной парадигме. Каждое выполнено по одному основанию деления — это и обеспечивает однородность групп.

18 Какие существуют правила идентификации элементов SC-кода? Какие требования часто забываются?

Правила:

- системный идентификатор — только латиница, цифры и знак подчёркивания; алфавит задан Стандартом OSTIS;
- системный идентификатор уникален в пределах базы знаний;
- префиксы по виду понятия: `concept_`, `nrel_`, `rrel_`, `section_`;
- у **неролевого** отношения русский идентификатор оканчивается знаком `*`;
- у **ролевого** — знаком `'`;
- естественно-языковые наименования задаются через `nrel_main_idtf` с обязательным указанием языка ссылки;
- принято правило «один файл — одно понятие», имя файла совпадает с системным идентификатором.

Часто забывают: звёздочку и апостроф в русских идентификаторах — а это единственное, что визуально отличает ролевое отношение от неролевого; указание `lang_ru` у ссылки; пишут кириллицу в системном идентификаторе; путают направление `=>` и `<=` у включения.

> **СКАЗАТЬ ВСЛУХ**
>
> Этот вопрос в методичке помечен как «какие требования часто забываются» — ответ именно про `*`, `'` и указание языка. И добавь: префиксы существуют ровно потому, что знаки `*` и `'` недопустимы в системном идентификаторе.

19 Приведите алгоритм формализации предметной области. На что обратить внимание?

- выбрать тему и **очертить границы**: что входит в область, что нет;
- выделить классы объектов исследования, разделить на максимальные и немаксимальные;
- выделить **исследуемые отношения**;
- описать спецификацию ПрО: `sc_node_struct` и три ролевых отношения;
- описать каждое понятие **отдельным файлом**: идентификаторы, примечание, пояснение, определение, источник;
- связать понятия **теоретико-множественными отношениями**;
- для отношений задать **свойства и домены** — первый, второй, область определения;
- описать **экземпляры**, минимум по одному на понятие, и связать их между собой;
- собрать базу знаний и проверить фрагменты в интерфейсе.

На что обратить внимание: однородность разбиений — одно основание на одно разбиение; направление дуг; непустота каждого класса; отсутствие висячих ссылок — всё упомянутое должно быть где-то описано.

Требования: не менее 30 понятий суммарно; теоретико-множественные отношения обязательны; у отношений свойства и домены; у понятий примечания, пояснения и определения; не менее одного экземпляра на понятие; экземпляры связаны между собой; источники открытые.

Разбор задач

Преподаватель даёт задачу одного из трёх типов. Важен метод, а не заученный ответ.

T1 Формализовать фактографическое высказывание

Метод:

- выписать все сущности из текста;
- для каждой определить класс — это абсолютные понятия;
- выписать связи между сущностями — это относительные понятия;
- выбрать вид отношения: связь бинарная и не является принадлежностью — неролевое `nrel_`; роль компонента внутри связки — ролевое `rrel_`;
- записать на SCs с идентификаторами и указанием языка ссылок.

Пример. «Караси — неприхотливые промысловые рыбы, тело высокое, чешуя крупная и гладкая.»

```
concept_crucian
<- sc_node_class;
<= nrel_inclusion: concept_commercial_fish;
=> nrel_main_idtf: [карась] (* <- lang_ru;; *);
=> nrel_body_shape: [высокое тело] (* <- lang_ru;; *);
=> nrel_scale_type: [крупная гладкая чешуя] (* <- lang_ru;; *);;
```

> СКАЗАТЬ ВСЛУХ
>
> Высказывание фактографическое, потому что в нём нет ни одной sc-переменной — только константы.

T2 Формализовать часть предметной области на заданную тему

Метод — тот же алгоритм, что в вопросе 19, но вслух и коротко:

- назвать максимальный класс объектов исследования;
- дать два-три подкласса и указать, включение это или разбиение;
- назвать два-три отношения и **сразу указать их домены** — это то, что чаще всего забывают;
- привести по экземпляру на класс и связать экземпляры отношением.

Пример для темы «Сотрудники университета»: максимальный класс — «сотрудник университета»; подклассы — «преподаватель», «сотрудник администрации»; отношения — «заведует кафедрой*» с первым доменом «преподаватель» и вторым «кафедра», «учёная степень*»; экземпляры — конкретный преподаватель и конкретная кафедра, связанные первым отношением.

ТЗ Устранить противоречия во фрагменте текста

Где искать:

- **цикл во включении** — A включает B, B включает A; нарушает антисимметричность;
- **несоответствие доменов** — связка построена между сущностями, не принадлежащими объявленным доменам отношения;
- **путаница класса и экземпляра** — сущность одновременно объявлена подклассом через `nrel_inclusion` и элементом через `<-` одного и того же класса;
- **нарушение разбиения** — подклассы пересекаются или не покрывают надкласс;
- **дублирование идентификаторов** — два элемента с одним системным идентификатором;
- **свойства отношения противоречат связкам** — отношение объявлено антирефлексивным, а есть связка элемента с самим собой.

> **СКАЗАТЬ ВСЛУХ**
> Порядок действий: сначала проверяю теоретико-множественные отношения на циклы, потом соответствие связок доменам, потом объявленные свойства отношений.

Синтаксис SCs под рукой

Если попросят написать фрагмент на доске.

ЗНАК	СМЫСЛ
`;;`	конец предложения
`;`	разделитель внутри предложения, субъект тот же
`<-` / `->`	принадлежность: элемент ← класс / класс → элемент
`<=` / `=>`	связка по неролевому отношению; `<=` — входящая, `=>` — исходящая
`(* … *)`	контур: описание предыдущего элемента
`[текст]`	sc-ссылка, файл с содержимым на естественном языке
`...`	анонимный узел
`= [* … *]`	содержимое структуры
`_имя`	sc-переменная
`/* */`	комментарий

Класс с надклассом и примечанием:

```
concept_compiler
<- sc_node_class;
<= nrel_inclusion: concept_programming_language_implementation;
=> nrel_main_idtf: [компилятор] (* <- lang_ru;; *);
=> nrel_note: [Программа, переводящая исходный текст
   с языка программирования в машинный код.]
   (* <- lang_ru;; *);;
```

Отношение с доменами:

```
nrel_language_author
<- sc_node_norole_relation;
<- binary_relation;
<- oriented_relation;
<- antireflexive_relation;
=> nrel_main_idtf: [автор языка программирования*]
   (* <- lang_ru;; *);
=> nrel_first_domain: concept_programming_language;
=> nrel_second_domain: concept_programming_language_developer;;
```

Если спросят про троеточия

В нашей ПрО 97 анонимных узлов — вопрос вероятен.

? Почему в базе знаний так много элементов `...` ?

Это **анонимные sc-элементы** — узлы без системного идентификатора. У нас их три вида: 40 узлов определений, 40 узлов перевода на естественный язык, 17 узлов области определения отношений.

Узел нужен там, где у сущности есть **собственные свойства**. Определение — самостоятельная сущность: у него своё имя «Опр. (компилятор)», свой тип `definition`, свой список используемых констант. Записать это прямо в понятие нельзя — получилось бы, что сам компилятор называется «Опр. (компилятор)» и сам является определением.

Имени таким узлам не дают: вне своего контекста они смысла не имеют и отдельно не разыскиваются.

> **СКАЗАТЬ ВСЛУХ**
>
> Контраст, который удобно показать: примечание `nrel_note` цепляется к понятию напрямую — у примечания нет собственных свойств, это просто текст. А определению узел нужен. Так же оформлены определения в шаблонах методички и в штатной базе знаний NIKA.

Предметная область: 23 абсолютных понятия, 17 относительных, 40 экземпляров, 70 связей, 77 файлов `.scs`. Каталог `kb/extra/section_subject_domain_of_programming_languages`, ветка `tpis-2023`.